Laboratorio 2 — Semáforos en XV6

Clase 4: cierre de `pingpong` y debugging

Sistemas Operativos — FaMAF, UNC

24/09/2026

Agenda

- Avisos
- Dónde tiene que estar cada grupo
- Debugging
- La entrega del 28/09

Avisos

Dos arreglos del kickstarter

Si bajaron el `.tgz` antes de esta semana, tienen que aplicarlos a mano. No les llegan solos.

1) `gradelib.py` (make grade no arranca en Python 3.13)

```
from pipes import quote      →   from shlex import quote
```

2) `user/semtest.c` (no compila aunque esté todo bien)

borrar estas tres líneas:

```
#ifndef sem_open
#define sem_open sem_init
#endif
```

Cómo se ven esos dos errores

Si ven esto	Es
`ModuleNotFoundError: No module named 'pipes'`	arreglo **1**
`implicit declaration of function 'sem_init'`	arreglo **2**

Ninguno de los dos es culpa de su código. El segundo además impide compilar todo, no sólo ese test: `_semtest` está en `UPROGS`, así que sin él no hay `fs.img` ni `make qemu`.

Dónde tiene que estar cada grupo

Meta del 28/09 23:59

- ☐ `kernel/semaphore.c` con las 4 operaciones.
- ☐ Syscalls registradas en los 6 lugares.
- ☐ `seminit()` llamado desde `main.c`.
- ☐ `user/pingpong.c` compilando y en `UPROGS`.
- ☐ `make grade-parcial` → **11/11**.
- ☐ Push al repo del grupo.

Quedan 4 días. Esta es la última clase antes de la entrega.

Si llegan atrasados, en este orden

1. Los **stubs** y las syscalls registradas (que compile).
2. `sem_open` y `sem_close` (la tabla ya funciona).
3. `sem_up` y `sem_down` (acá aparece `sleep`/`wakeup`).
4. `pingpong`.

Cada paso deja algo verificable. No arranquen por `pingpong`: sin el kernel no hay nada que probar.

Repaso: las cuatro syscalls

Syscall	**Éxito**	**Error (devuelve 0)**
`sem_open`(id, val)	devuelve `id`	id $\notin 1..64$; ya abierto; val < 0
`sem_close`(id)	devuelve `id`	id no abierto
`sem_down`(id)	devuelve `id`	no abierto; cerrado mientras esperaba
`sem_up`(id)	devuelve `id`	no abierto; llegó a `SEM_VALUE_MAX`

Repaso: los invariantes

1. `value` nunca negativo — esperar es dormir, no dejar el contador en -1.
2. `sem_close` marca cerrado **y** hace `wakeup`; los que esperaban devuelven **0**.
3. `sem_up` **sin tope** en el valor inicial.
4. Todo se toca con el lock de `sem[id]` tomado — incluida la validación.
5. `sleep(&sem[id], &sem[id].lk)` adentro de un `while`, nunca de un `if`.

Debugging

Síntoma → causa: el kernel

Síntoma	**Causa probable**
QEMU se cuelga y no vuelve	un `sem_down` sin su `sem_up`; o `sleep` con el lock equivocado
`panic: acquire`	tomaron el mismo lock dos veces
`panic: sched locks`	se durmieron con un spinlock tomado
Todo devuelve 0 siempre	los stubs sin reemplazar
Anda con 1 proceso, falla con varios	validación **fuera** del lock

Síntoma → causa: `pingpong`

Síntoma	**Causa probable**
Empieza con `pong`	valores iniciales invertidos
A veces salen dos `ping` seguidos	`if` en lugar de `while` (el contador quedó negativo)
Anda solo, falla el test de concurrencia	ids hardcodeados
Se cuelga con N grande	falta un `sem_up` en algún camino
Sale bien pero el prompt no vuelve	falta `wait(0)`

Los 6 tests de `semtest`

Test	Qué mira
`noinit`	usar un semáforo que nunca se abrió
`init`	que el valor inicial sea el que pidieron
`closeuse`	usar uno ya cerrado
`exhaust`	quedarse sin ids, y reusar tras cerrar
`grow`	que `sem_up` pase el valor inicial
`reopen`	cerrar y volver a abrir el mismo id

6 puntos de `semtest` + 5 de `pingpong` = 11

Técnicas

```
Ctrl-p            dentro de xv6: lista los procesos y su estado
                  (si ven varios en sleep, es un deadlock)

Ctrl-a  x         salir de QEMU
```

```
// trazas desde user space, sin tocar el kernel
printf("pid=%d antes del down\n", getpid());
sem_down(1);
printf("pid=%d despues del down\n", getpid());
```

Antes de pedir ayuda

1. ¿Qué test falla? `make grade-parcial` lo dice.
2. ¿Pasa siempre o a veces? (a veces = carrera; siempre = lógica)
3. ¿Anda con un proceso y falla con varios? (mirá los locks)
4. ¿Cuelga o devuelve mal? (cuelga = `sleep`; mal = validación)

Con esas cuatro respuestas, el problema casi siempre queda en dos o tres líneas.

La entrega

Calendario

Fecha	Qué pasa
15/09 – 24/09	Clases 1–4: xv6, syscalls, `pingpong`
28/09	**Entrega intermedia** — `grade-parcial` **11/11**
29/09	Clase 5 — arranca `prodcons`
01/10	Clase 6 — implementación de `prodcons`
06/10	Clase 7 — cierre
07/10	**Entrega final — 15/15** + video

Para esta semana

1. Cerrar `make grade-parcial` $\rightarrow$ **11/11**.
2. `make clean` y push al repo del grupo.
3. Recién después, mirar la sección `prodcons` del enunciado.

Entrega intermedia: 28/09 23:59